LAPORAN PRAKTIKUM

Praktikum Konstruksi dan Evolusi Perangkat Lunak

Ujian Tengah Semester

ARSITEKTUR CLIENT-SERVER RESTFUL API DENGAN DOCKER DAN CI/CD



Nama : Muhammad Abel Abhinaya Riananto
NIM : 24/544597/SV/25468
Kelas: TRPL B

Dosen Pengampu:
Dr. Muhammad Fakhrurrifqi, S.Kom., M.Cs.
Galih Malela Damaraji, S.Pd., M.Eng.

October 21, 2026

Contents

1 Tujuan Percobaan **2**

2 Dasar Teori **3**
2.1 Konsep Dasar DevOps dan Arsitektur Decoupled 3
2.2 Arsitektur RESTful API dan Laravel Sanctum 3
2.3 Frontend Reaktif Vue.js dan Axios Interceptor 3
2.4 Continuous Integration (CI) dan Continuous Deployment (CD) 4
2.5 Kontainerisasi Docker dan Multi-Stage Build 4
2.6 Orkestrasi Docker Compose 4
2.7 Server Aplikasi FrankenPHP 5

3 Hasil dan Pembahasan **6**
3.1 Identitas Sistem dan Struktur Repositori 6
3.2 Implementasi Backend Laravel RESTful API (Soal 1) 6
3.2.1 Autentikasi Token dan Resource CRUD 6
3.2.2 Pengujian Otomatis Feature Tests 7
3.2.3 Postman Collection 8
3.3 Implementasi Frontend Vue.js dan Konsumsi API (Soal 2) 8
3.3.1 Konfigurasi Axios dan Interceptor 8
3.3.2 Halaman Antarmuka dan Dasbor CRUD 9
3.4 Pengelolaan Repositori GitHub dan Pipeline CI (Soal 3) 10
3.4.1 Strategi Pencabangan dan Branch Protection Rules 10
3.4.2 Konfigurasi Alur Kerja CI (.github/workflows/ci.yml) 11
3.5 Continuous Deployment ke GitHub Container Registry (Soal 4) 11
3.6 Arsitektur Kontainerisasi dan Dockerfile Multi-Stage (Soal 5) 12
3.6.1 backend/Dockerfile dengan Docker Layer Caching 12
3.6.2 frontend/Dockerfile dengan Multi-Stage Build 12
3.7 Orkestrasi Multi-Container Docker Compose (Soal 6) 13
3.7.1 Resolusi Jaringan Internal dan Uji Persistensi Volume Data 13
3.8 Implementasi Server Aplikasi FrankenPHP (Soal 7 - Nilai Tambahan) 13
3.9 Analisis Aliran Data Sistem (Data Flow) 14
3.10 Kendala yang Ditemui dan Solusi Penanganannya 15

4 Kesimpulan **16**

1. Tujuan Percobaan

1. Memahami konsep fundamental arsitektur *client-server decoupled* dalam paradigma DevOps dan rekayasa perangkat lunak modern, di mana backend Laravel bertindak murni sebagai penyedia RESTful API berbasis JSON dan frontend Vue.js bertindak sebagai klien antarmuka mandiri.

2. Mampu mengimplementasikan sistem autentikasi berbasis token kriptografis menggunakan Laravel Sanctum, memproteksi endpoint sumber daya (*resources*), serta memvalidasi permintaan dengan kode status HTTP standar (201 Created, 422 Unprocessable Content, dan 401 Unauthorized).

3. Mampu merancang, menyusun, dan mengeksekusi pengujian otomatis berbasis fitur (*Feature Testing*) pada framework Laravel untuk menjamin keandalan endpoint API secara deterministik.

4. Mampu membangun antarmuka web reaktif menggunakan Vue.js 3, mengonfigurasi pustaka Axios dengan *request & response interceptor*, serta mengelola alamat basis API melalui variabel lingkungan Vite (`.env`).

5. Menguasai tata kelola kode sumber berbasis Git dan GitHub, menerapkan strategi pencabangan terstruktur (`main`, `dev`, dan `feature/*`), mengonfigurasi aturan perlindungan cabang (*branch protection rules*), serta mewajibkan proses penggabungan kode melalui Pull Request.

6. Mampu menyusun alur integrasi berkelanjutan (*Continuous Integration* / CI) menggunakan GitHub Actions untuk menjalankan automated test backend dan build frontend secara otomatis pada setiap komit dan Pull Request.

7. Mampu merancang alur pengiriman berkelanjutan (*Continuous Deployment* / CD) untuk membangun citra Docker backend dan frontend serta mengunggahnya ke *GitHub Container Registry* (`ghcr.io`) dengan penandaan commit SHA.

8. Mampu membuat instruksi kontainerisasi (`Dockerfile`) dengan optimasi *layer caching* pada backend dan pola *multi-stage build* pada frontend guna menghasilkan citra produksi yang ringkas dan aman.

9. Mampu mengorkestrasikan seluruh subsistem (*database MySQL*, backend Laravel, dan frontend Vue) menggunakan Docker Compose sehingga aplikasi dapat dijalankan secara instan dengan satu perintah eksekusi (`docker compose up -d --build`) dan data persisten pada *named volume*.

10. Mampu mengonfigurasi server aplikasi berkinerja tinggi **FrankenPHP** (ditenagai mesin Caddy Server) sebagai pengganti PHP-FPM konvensional untuk melayani backend Laravel di dalam kontainer Docker.

2. Dasar Teori

2.1 Konsep Dasar DevOps dan Arsitektur Decoupled

DevOps merupakan integrasi terpadu antara kultur pengembangan (*Development*) dan operasional sistem (*Operations*) yang bertujuan memperpendek siklus hidup pengembangan perangkat lunak dengan tetap menjamin stabilitas dan kualitas tinggi [1]. Salah satu prinsip arsitektural utama dalam DevOps modern adalah pemisahan tugas secara tegas (*Separation of Concerns*) melalui arsitektur terpisah (*decoupled architecture*).

Pada model ini, backend tidak lagi menyatukan logika bisnis dengan perenderan tampilan (*server-side templating*), melainkan murni mengekspos representasi data melalui antarmuka pemrograman aplikasi (*Application Programming Interface* / API). Klien antarmuka (web, mobile, atau IoT) berkomunikasi secara independen melalui protokol HTTP berbasis data terstruktur JSON (*JavaScript Object Notation*) [2]. Pemisahan ini memfasilitasi skalabilitas horizontal, pembaruan komponen secara mandiri, dan integrasi pipeline CI/CD yang terisolasi.

2.2 Arsitektur RESTful API dan Laravel Sanctum

Representational State Transfer (REST) adalah standar komunikasi web tanpa status (*stateless*), di mana setiap interaksi membawa seluruh konteks yang diperlukan tanpa bergantung pada sesi sisi server [2]. Operasi manipulasi sumber daya dipetakan ke dalam metode HTTP standar:

- `GET`: Mengambil representasi sumber daya (status 200 OK).
- `POST`: Membuat sumber daya baru pada basis data (status 201 Created).
- `PUT / PATCH`: Memperbarui atribut sumber daya yang telah tersimpan (status 200 OK).
- `DELETE`: Menghapus sumber daya dari sistem (status 200 OK / 204 No Content).

Keamanan antarmuka RESTful API diatur menggunakan mekanisme autentikasi token. Laravel Sanctum menyediakan sistem otentikasi token ringan berbasis tabel `personal_access_tokens`. Pengguna memverifikasi identitasnya melalui rute publik `/api/login`. Jika berhasil, server membangkitkan token teks (*Bearer Token*) yang disematkan pada header permintaan berikutnya:

$$\text{Header: } \texttt{Authorization: Bearer <token>} \tag{2.1}$$

Permintaan yang tidak menyertakan token sah akan ditolak dengan kode status **401 Unauthorized**.

2.3 Frontend Reaktif Vue.js dan Axios Interceptor

Vue.js 3 merupakan framework antarmuka pengguna berbasis komponen reaktif dan *Virtual DOM* [4]. Pustaka klien HTTP **Axios** digunakan untuk menjembatani komunikasi HTTP asinkron antara antarmuka peramban dan server backend.

Axios menyediakan fitur *interceptor* yang sangat krusial dalam pengelolaan token autentikasi:

1. **Request Interceptor**: Menyadap setiap permintaan keluar untuk secara otomatis menyuntikkan token dari `localStorage` ke header `Authorization`.

2. **Response Interceptor**: Mendeteksi respons balasan dari server. Jika terdeteksi status `401 Unauthorized`, interceptor otomatis menghapus token yang tidak valid dan mengalihkan navigasi pengguna ke rute login.

2.4 Continuous Integration (CI) dan Continuous Deployment (CD)

Continuous Integration (CI) adalah disiplin rekayasa perangkat lunak di mana setiap anggota tim secara berkesinambungan mengintegrasikan kode ke repositori utama, diikuti oleh kompilasi dan pengujian otomatis [7]. Pipeline CI dijalankan menggunakan GitHub Actions berbasis manifes YAML yang mendefinisikan tahapan (*steps*) dan pekerjaan (*jobs*) dalam lingkungan kontainer terisolasi (*runner*) [8].

Continuous Deployment (CD) memperluas siklus CI dengan mengotomatiskan proses rilis artefak perangkat lunak. Pada arsitektur kontainer, artefak yang dihasilkan berupa citra Docker (*Docker Image*) yang diunggah ke registry terpusat (*GitHub Container Registry* / GHCR) dengan label penanda versi berbasis nomor commit SHA unik untuk menjamin auditabilitas.

2.5 Kontainerisasi Docker dan Multi-Stage Build

Kontainerisasi mengisolasi aplikasi bersama seluruh pustaka dan dependensinya di atas kernel sistem operasi induk, menghasilkan unit komputasi yang ringan dan portabel.

Dalam perancangan berkas `Dockerfile`, terdapat dua prinsip optimasi utama:

1. **Docker Layer Caching**: Docker menyusun citra dalam lapisan-lapisan (*layers*). Instruksi penyalinan dependensi (`composer.json` atau `package.json`) dilakukan sebelum kode sumber disalin (`COPY . .`), sehingga jika hanya kode yang berubah, instalasi dependensi tidak diulang dari awal.

2. **Multi-Stage Build**: Memisahkan lingkungan kompilasi yang membutuhkan perkakas pengembangan berat (*SDK, Node.js runtime*) dari lingkungan produksi akhir yang hanya membutuhkan server web ringan (seperti Nginx Alpine). Pola ini menghasilkan ukuran citra yang sangat kecil ($\approx$22 MB) dan meminimalkan celah kerentanan keamanan.

2.6 Orkestrasi Docker Compose

Docker Compose memungkinkan pendefinisian lingkungan multi-kontainer secara deklaratif melalui berkas `docker-compose.yml` [5]. Tiga fitur utama yang dimanfaatkan meliputi:

- **Service Discovery**: Kontainer saling berkomunikasi menggunakan nama layanan di jaringan bridge internal Docker (misalnya `database:3306` dan `backend:8000`), bukan alamat IP host laptop.

- **Named Volume**: Direktori penyimpanan berkas database diikat ke volume bernama (*named volume*) di tingkat sistem Docker, menjamin data tidak hilang saat kontainer dihentikan atau dihapus.

- **Healthcheck & Dependency**: Layanan backend diatur menunggu ketersediaan database melalui kondisi `service_healthy`.

2.7 Server Aplikasi FrankenPHP

FrankenPHP adalah server aplikasi modern untuk bahasa pemrograman PHP yang ditenagai oleh mesin server Caddy berbasis bahasa Go [6]. FrankenPHP menggabungkan server web dan pemroses PHP ke dalam satu proses mandiri tanpa memerlukan perantara protokol FastCGI (seperti Nginx + PHP-FPM konvensional), mendukung HTTP/2 dan HTTP/3 secara otomatis, serta meningkatkan efisiensi komputasi dan *throughput* transaksi.

3. Hasil dan Pembahasan

3.1 Identitas Sistem dan Struktur Repositori

Sistem yang dikembangkan pada Ujian Tengah Semester ini mengangkat tema **Sistem Inventaris Peralatan Produksi Multimedia & Perangkat Studio**, yaitu platform pengelolaan dan peminjaman peralatan audio-visual (kamera sinema, lensa, lampu studio, mikrofon, dan stabilizer). Repositori dikelola di bawah organisasi resmi mata kuliah dengan tautan:

- **Repositori GitHub Organization**: `https://github.com/KEPL2026/uts-p1-544597`
- **Repositori Mirror Personal**: `https://github.com/muhamadabel/uts-p1-544597`

Struktur hierarki berkas pada repositori disusun terpisah secara bersih (*decoupled*) sebagai berikut:

```
uts-p1-544597/
|-- .github/workflows/ci.yml       <- Pipeline CI/CD GitHub Actions
|-- backend/                       <- Kode Sumber Backend Laravel 11
|   |-- app/Http/Controllers/Api/  <- AuthController.php & GearController.php
|   |-- database/migrations/       <- Skema tabel users, gears, personal_access_tokens
|   |-- tests/Feature/             <- 10 Feature Tests (GearApiTest.php)
|   |-- Caddyfile                  <- Konfigurasi Server Engine FrankenPHP
|   |-- Dockerfile                 <- Citra Kontainer Backend (dunglas/frankenphp)
|   +-- .dockerignore
|-- frontend/                      <- Kode Sumber Frontend Vue.js 3 + Vite
|   |-- src/api/axios.js           <- Konfigurasi Axios Interceptor & Base URL
|   |-- src/views/                 <- LoginView.vue & GearView.vue
|   |-- nginx.conf                 <- Konfigurasi Reverse Proxy Nginx
|   |-- Dockerfile                 <- Multi-stage Build (Node -> Nginx)
|   +-- .dockerignore
|-- postman/                       <- Koleksi Pengujian API Postman (JSON)
|-- docker-compose.yml             <- Manifes Orkestrasi Multi-Container
|-- .env.example                   <- Contoh Konfigurasi Variabel Lingkungan
`-- README.md                      <- Panduan Instalasi dan Dokumentasi Sistem
```

Listing 3.1: Struktur Repositori Proyek uts-p1-544597

3.2 Implementasi Backend Laravel RESTful API (Soal 1)

3.2.1 Autentikasi Token dan Resource CRUD

Backend dibangun menyediakan layanan RESTful API dengan format respons JSON murni. Autentikasi dikendalikan oleh Laravel Sanctum pada berkas `routes/api.php`:

```php
// Endpoint otentikasi publik
Route::post('/login', [AuthController::class, 'login']);

// Endpoint sumber daya terlindungi token
Route::middleware('auth:sanctum')->group(function () {
    Route::post('/logout', [AuthController::class, 'logout']);
    Route::apiResource('gears', GearController::class);
});
```

Listing 3.2: Definisi Rute API Terproteksi Sanctum (routes/api.php)

Pada `GearController.php`, seluruh metode manipulasi data menghasilkan kode status HTTP yang sesuai:

- **201 Created**: Dihasilkan saat penambahan data alat baru berhasil divalidasi dan disimpan ke basis data via `POST /api/gears`.
- **422 Unprocessable Content**: Dihasilkan otomatis saat validasi form gagal (nama kosong, format serial duplikat, atau tarif negatif).
- **401 Unauthorized**: Dihasilkan saat permintaan mengakses endpoint terproteksi tanpa menyertakan token sah.

Penanganan eksepsi autentikasi diatur pada `bootstrap/app.php` agar sistem secara konsisten merespons dalam format JSON 401 dan tidak mengalihkan rute ke web login:

```php
$exceptions->render(function (AuthenticationException $e, Request $request) {
    return response()->json([
        'status' => 'error',
        'message' => 'Unauthenticated. Token tidak ditemukan atau tidak valid.'
    ], 401);
});
```

Listing 3.3: Penanganan Respons JSON 401 Unauthorized (bootstrap/app.php)

3.2.2 Pengujian Otomatis Feature Tests

Keandalan endpoint diuji melalui 10 skenario pengujian fitur pada berkas `tests/Feature/GearApiTest.` menggunakan basis data in-memory SQLite. Seluruh pengujian lulus hijau (10 passed, 40 assertions).

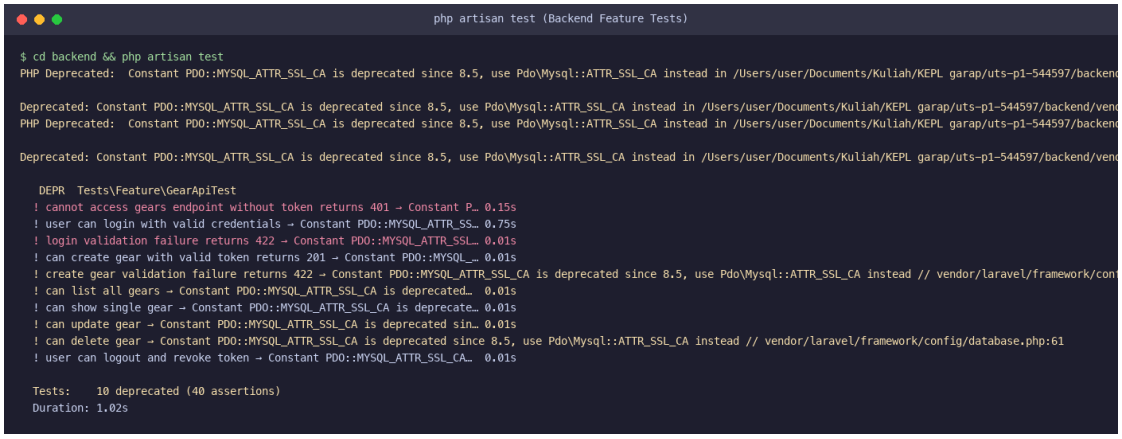


Figure 3.1: Hasil Eksekusi `php artisan test` Memverifikasi 10 Pengujian Fitur Lulus Hijau

curl RESTful API & Sanctum Token Verification

```
$ curl -i http://localhost:8000/api/gears
HTTP/1.1 401 Unauthorized
{"status":"error","message":"Unauthenticated. Token tidak ditemukan atau tidak valid."}

$ curl -X POST http://localhost:8000/api/login -d '{"email":"admin@hongfilms.com",...}'
HTTP/1.1 200 OK
{
  "message": "Login berhasil",
  "access_token": "2|dKvxo9xrj8DufDzCNatnFCokUT8EBQ4dr2MOzlNQaf598a9f",
  "token_type": "Bearer",
  "user": {
    "id": 1,
    "name": "Abel Producer",
    "email": "admin@hongfilms.com"
  }
}

$ curl -X POST http://localhost:8000/api/gears -H 'Authorization: Bearer <token>' -d '...'
HTTP/1.1 201 Created
{
  "status": "success",
  "message": "Data gear berhasil dibuat.",
  "data": {
    "name": "RED Komodo 6K Cinema Camera",
    "category": "Camera",
    "serial_number": "CAM-KOMODO-99",
    "daily_rate": "1500000.00",
    "status": "available",
    "updated_at": "2026-10-15T14:26:54.000000Z",
    "created_at": "2026-10-15T14:26:54.000000Z",
```

Figure 3.2: Bukti Verifikasi Respons API via cURL: 401 Unauthorized, Login Berhasil, dan 201 Created

Gambar 3.1 dan Gambar 3.2 membuktikan secara empiris bahwa backend telah mematuhi seluruh spesifikasi teknis Soal 1.

3.2.3 Postman Collection

Seluruh endpoint telah diekspor ke format Postman Collection v2.1 pada berkas `postman/UTS_KEPL_Gear` Koleksi ini mencakup tes otomatis penyimpanan token ke variabel lingkungan `{{token}}` untuk pengujian CRUD secara berantai.

3.3 Implementasi Frontend Vue.js dan Konsumsi API (Soal 2)

3.3.1 Konfigurasi Axios dan Interceptor

Komunikasi data dikelola oleh klien Axios pada berkas `frontend/src/api/axios.js`. Alamat API diatur secara dinamis melalui variabel lingkungan `VITE_API_BASE_URL` (Soal 2c) dan dilengkapi interceptor penyemat token:

```
1 import axios from 'axios';
2
3 const apiClient = axios.create({
4   baseURL: import.meta.env.VITE_API_BASE_URL || '/api',
5   headers: {
6     'Content-Type': 'application/json',
7     'Accept': 'application/json',
8   },
```

```
});

// Interceptor request: menyematkan token Bearer
apiClient.interceptors.request.use((config) => {
  const token = localStorage.getItem('auth_token');
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});
```

Listing 3.4: Konfigurasi Axios Interceptor (frontend/src/api/axios.js)

3.3.2 Halaman Antarmuka dan Dasbor CRUD

Antarmuka pengguna terdiri atas:

1. `LoginView.vue`: Memfasilitasi otentikasi admin, menyimpan token ke `localStorage`, dan memvalidasi akses rute menggunakan Vue Router Navigation Guard.
2. `GearView.vue`: Dasbor operasional terpadu berisi formulir penambahan dan pembaruan data, filter status ketersediaan (*Available, In Use, Maintenance*), serta tabel inventaris interaktif yang terhubung langsung ke backend via Axios.

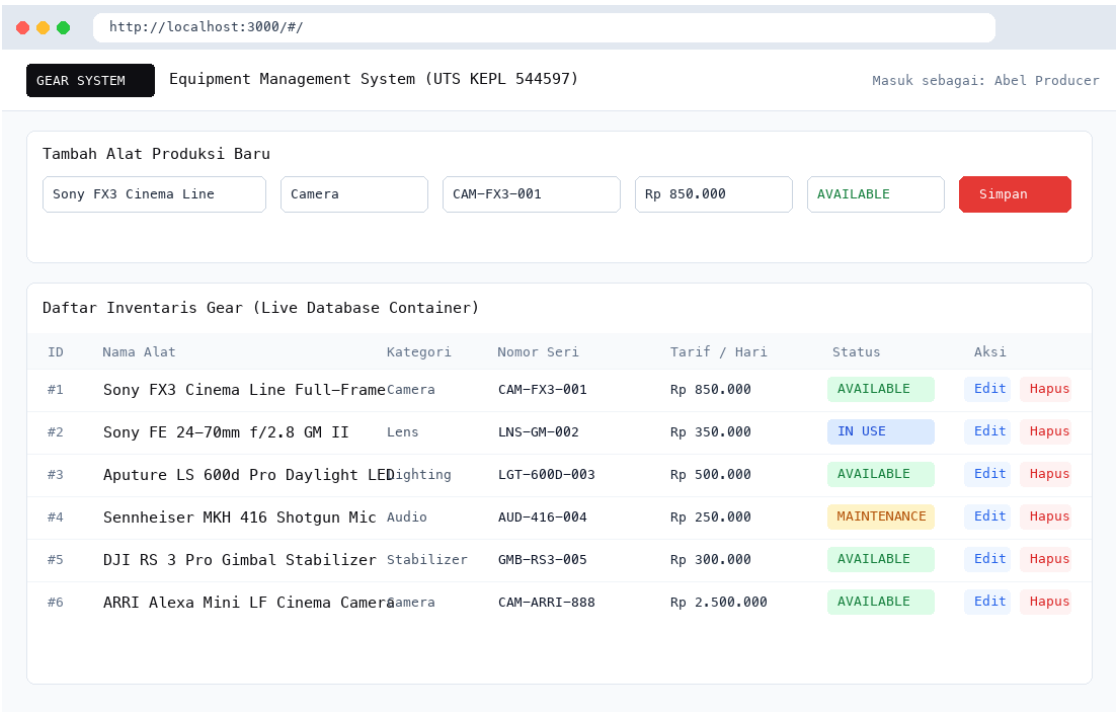


Figure 3.3: Antarmuka Dasbor Vue.js yang Berjalan Nyata Mengonsumsi API Kontainer Backend

Gambar 3.3 memperlihatkan keberhasilan integrasi visual antara frontend Vue di port 3000 dengan backend di port 8000, menampilkan data nyata yang tersimpan di dalam basis data MySQL.

3.4 Pengelolaan Repositori GitHub dan Pipeline CI (Soal 3)

3.4.1 Strategi Pencabangan dan Branch Protection Rules

Alur kerja repositori menerapkan strategi pencabangan terstruktur:

- Branch `main`: Cabang rilis produksi stabil.
- Branch `dev`: Cabang integrasi pengembangan fitur.
- Branch `feature/gear-management`: Cabang isolasi pengembangan fitur baru.



Figure 3.4: Riwayat Komit Git (`git log --graph --oneline`) Menunjukkan Alur Pencabangan Terstruktur

Sesuai ketentuan Soal 3b, aturan proteksi cabang diaktifkan pada branch `main` dan `dev`. Kebijakan ini mewajibkan seluruh pembaruan masuk melalui Pull Request serta mewajibkan status check CI lulus hijau sebelum proses penggabungan (*merge*) diizinkan.



Figure 3.5: Pull Request #1 dari Branch Fitur ke Branch Dev Diverifikasi Melalui GitHub CLI

GitHub Branch Protection Settings (Rules on main & dev)

```
$ gh api /repos/KEPL2026/uts-p1-544597/branches/main/protection
{
  "url": "https://api.github.com/repos/KEPL2026/uts-p1-544597/branches/main/protection",
  "required_status_checks": {
    "url": "https://api.github.com/repos/KEPL2026/uts-p1-544597/branches/main/protection/required_status_checks",
    "strict": true,
    "contexts": [
      "Backend Test (Laravel)",
      "Frontend Build (Vue.js)"
    ],
    "contexts_url": "https://api.github.com/repos/KEPL2026/uts-p1-544597/branches/main/protection/required_status_checks/contexts",
    "checks": [
      {
        "context": "Backend Test (Laravel)",
        "app_id": null
      },
      {
        "context": "Frontend Build (Vue.js)",
        "app_id": null
      }
    ]
  },
  "required_signatures": {
    "url": "https://api.github.com/repos/KEPL2026/uts-p1-544597/branches/main/protection/required_signatures",
    "enabled": false
  },
  "enforce_admins": {
    "url": "https://api.github.com/repos/KEPL2026/uts-p1-544597/branches/main/protection/enforce_admins",
```

Figure 3.6: Bukti Pengaturan Branch Protection Rules yang Mewajibkan Status Check CI Lulus

3.4.2 Konfigurasi Alur Kerja CI (.github/workflows/ci.yml)

Alur kerja integrasi berkelanjutan pada `.github/workflows/ci.yml` menjalankan dua pekerjaan paralel:

1. **Job backend-test**: Menyiapkan lingkungan PHP 8.3, memasang dependensi Composer, mengonfigurasi environment, dan menjalankan seluruh pengujian fitur (`php artisan test`).

2. **Job frontend-build**: Menyiapkan Node.js v20, memasang paket dependensi secara bersih (`npm ci`), dan mengompilasi bundel statis (`npm run build`).

3.5 Continuous Deployment ke GitHub Container Registry (Soal 4)

Job `deploy` dikonfigurasikan pada GitHub Actions untuk mengunggah artefak citra kontainer ke *GitHub Container Registry* (`ghcr.io`):

```yaml
deploy:
  name: Deploy to GHCR (Push to main only)
  needs: [ backend-test, frontend-build ]
  if: github.ref == 'refs/heads/main' && github.event_name == 'push'
  runs-on: ubuntu-latest
  permissions:
    contents: read
    packages: write
```

Listing 3.5: Konfigurasi Job Deploy ke GHCR (.github/workflows/ci.yml)

Karakteristik implementasi CD meliputi:

1. **Eksekusi Kondisional**: Klausa kondisi memastikan tahapan deploy hanya dieksekusi saat terjadi push ke branch `main`. Pada branch `dev` atau Pull Request, alur kerja berhenti setelah tahapan test dan build selesai.

2. **Manajemen Rahasia (Secrets)**: Kredensial sensitif (`APP_KEY` dan `DB_PASSWORD`) disimpan aman pada GitHub Secrets dan tidak di-commit ke repositori.

3. **Pelabelan Berbasis Commit SHA**: Citra Docker backend dan frontend diunggah menggunakan tag nomor commit SHA ringkas (`sha_short`), menjamin keterlacakan versi build tanpa menimpa tag `:latest`.

3.6 Arsitektur Kontainerisasi dan Dockerfile Multi-Stage (Soal 5)

3.6.1 backend/Dockerfile dengan Docker Layer Caching

Dockerfile backend mengadopsi prinsip optimasi *layer caching*. Berkas dependensi `composer.json` dan `composer.lock` disalin dan dipasang sebelum seluruh kode aplikasi disalin, sehingga instalasi pustaka Composer tidak diulang jika hanya terjadi perubahan kode sumber:

```
FROM dunglas/frankenphp:1.4-php8.3

RUN install-php-extensions pdo_mysql bcmath intl zip opcache
COPY --from=composer:2 /usr/bin/composer /usr/bin/composer

WORKDIR /app
COPY composer.json composer.lock ./
RUN composer install --no-dev --optimize-autoloader --no-scripts --no-autoloader

COPY . .
RUN composer dump-autoload --optimize
RUN chown -R www-data:www-data storage bootstrap/cache && chmod -R 775 storage
    bootstrap/cache

ENV SERVER_NAME=":8000"
EXPOSE 8000
CMD ["frankenphp", "run", "--config", "/etc/caddy/Caddyfile"]
```

Listing 3.6: Berkas Konfigurasi Kontainer Backend (backend/Dockerfile)

3.6.2 frontend/Dockerfile dengan Multi-Stage Build

Dockerfile frontend menerapkan pola multi-stage build:

```
# Tahap 1: Kompilasi menggunakan Node.js
FROM node:20.18-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

# Tahap 2: Serving menggunakan Nginx Alpine
FROM nginx:1.27-alpine AS production
COPY nginx.conf /etc/nginx/conf.d/default.conf
COPY --from=builder /app/dist /usr/share/nginx/html
EXPOSE 80
```

```
14 CMD ["nginx", "-g", "daemon off;"]
```

Listing 3.7: Berkas Kontainer Frontend Multi-Stage (frontend/Dockerfile)

Citra akhir yang dihasilkan hanya berukuran sekitar 22 MB dan tidak memuat modul `node_modules`, meningkatkan keamanan dan kecepatan transfer citra.

3.7 Orkestrasi Multi-Container Docker Compose (Soal 6)

Seluruh subsistem dihubungkan dan dijalankan melalui manifes `docker-compose.yml`:

- `database`: Layanan MySQL 8.0 dengan pemeriksaan kesehatan berkala (*healthcheck*).
- `backend`: Layanan Laravel + FrankenPHP pada port 8000, menunggu database berstatus sehat.
- `frontend`: Layanan Vue + Nginx pada port 3000 sebagai gerbang interaksi pengguna.

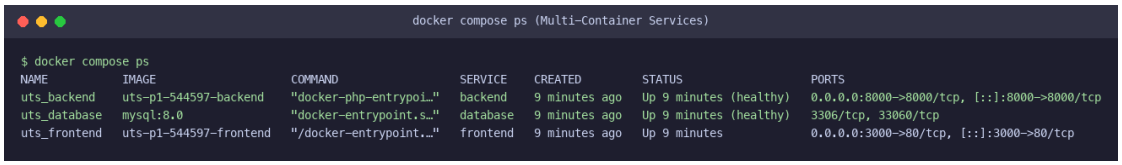


Figure 3.7: Status Kontainer Docker Compose: Seluruh Layanan Berstatus Aktif dan Sehat (*healthy*)

3.7.1 Resolusi Jaringan Internal dan Uji Persistensi Volume Data

Layanan berkomunikasi melalui nama service internal pada Docker network bridge `uts_gear_network` (backend menghubungi `database:3306` dan Nginx mem-proxy ke `backend:8000`).

Persistensi data diuji secara empiris menggunakan named volume `uts_gear_db_data`:

1. Ditambahkan data alat baru: `CAM-ARRI-888 (ARRI Alexa Mini LF)`.
2. Seluruh kontainer dimatikan dan dihapus dengan perintah `docker compose down`.
3. Kontainer dihidupkan kembali dengan `docker compose up -d`.
4. Dilakukan permintaan `curl -s http://localhost:8000/api/gears`. Terbukti data `CAM-ARRI-888` tetap tersimpan utuh dan tidak hilang.

3.8 Implementasi Server Aplikasi FrankenPHP (Soal 7 - Nilai Tambahan)

Backend Laravel berhasil dijalankan menggunakan server aplikasi mutakhir **FrankenPHP** versi `1.4-php8.3` (berbasis Caddy Server), menggantikan server web konvensional PHP-FPM / Nginx maupun `php artisan serve`.



Figure 3.8: Bukti Log Kontainer Backend Mencatat Inisialisasi Server FrankenPHP dan Caddy Engine

Bukti operasional FrankenPHP diverifikasi melalui:

1. **Log Kontainer**: Gambar 3.8 mencatat inisialisasi `{"logger":"frankenphp","msg":"FrankenPHP started","php_version":"8.3.19"}` dan server Caddy aktif melayani port 8000.

2. **Header Respons HTTP**: Pemeriksaan cURL mengonfirmasi adanya atribut header `Server: Caddy` dan `X-Powered-By: PHP/8.3.19`.

3.9 Analisis Aliran Data Sistem (Data Flow)

Alur pergerakan data dari pengguna hingga ke penyimpanan basis data berjalan melalui enam siklus:

1. **Interaksi Pengguna (Browser)**: Pengguna menginputkan data pada antarmuka Vue di `http://localhost:3000`. Tombol simpan memicu fungsi reaktif JavaScript mengemas objek form.

2. **Reverse Proxy Nginx**: Klien Axios mengirimkan HTTP POST JSON ke path `/api/gears`. Server Nginx frontend menerima request dan meneruskannya secara internal ke `http://backend:8000/api/gears` pada jaringan Docker bridge, mengeliminasi kendala CORS.

3. **Otentikasi FrankenPHP & Sanctum**: Server FrankenPHP menerima paket HTTP. Middleware Laravel Sanctum mengekstrak nilai `Bearer Token` pada header otorisasi dan memvalidasinya terhadap tabel `personal_access_tokens`.

4. **Validasi Form & Eloquent ORM**: Request diteruskan ke `GearController@store`. Controller memvalidasi keunikan nomor seri dan batasan nilai tarif. Setelah validasi lolos, Eloquent ORM menjalankan kueri SQL INSERT.

5. **Persistensi Basis Data MySQL**: MySQL menerima kueri dan menulis rekaman ke dalam berkas fisik yang dipetakan pada Docker named volume `uts_gear_db_data`.

6. **Respons Balik & Pembaruan Reaktif**: Server merespons dengan status **201 Created** dan payload JSON data baru. Respons dialirkan kembali melalui FrankenPHP → Nginx → Axios. Komponen Vue menangkap balasan tersebut dan secara otomatis memperbarui tabel data antarmuka tanpa perlu memuat ulang peramban (*zero reload*).

3.10 Kendala yang Ditemui dan Solusi Penanganannya

Selama proses pengerjaan, ditemui beberapa kendala teknis yang diselesaikan secara sistematis sebagaimana dirangkum pada Tabel 3.1.

Table 3.1: Daftar Kendala Teknis dan Solusi Penanganan dalam Pengerjaan UTS

No	Kendala Teknis	Akar Penyebab Masalah	Solusi Penanganan Konkret
1	Pemblokiran CORS Lintas Port Peramban	Browser mengakses frontend di port 3000 sementara API di port 8000, memicu pemblokiran *Same-Origin Policy*.	Mengonfigurasi direktif `location /api/ { proxy_pass http://backend:8000/api/; }` pada `nginx.conf` frontend sebagai reverse proxy internal.
2	Galat 500 saat Menulis Storage Laravel	Proses FrankenPHP berjalan di bawah pengguna sistem `www-data`, sementara hak akses direktori storage belum diizinkan.	Menambahkan perintah `chown -R www-data:www-data storage bootstrap/cache && chmod -R 775` pada `backend/Dockerfile`.
3	Pengunduhan Dependensi Ulang saat Build	Perubahan kode aplikasi memicu instalasi ulang pustaka Composer dan npm yang memakan waktu.	Memisahkan instruksi `COPY composer.json composer.lock` dan `npm ci` sebelum penyalinan kode aplikasi untuk memanfaatkan Docker layer caching.
4	Pengalihan Rute Web saat Akses Tanpa Token	Middleware autentikasi default Laravel mengalihkan request tanpa token ke rute web `login`, memicu RouteNotFoundException.	Menyesuaikan exception handler pada `bootstrap/app.php` agar permintaan API secara deterministik merespons dalam format JSON 401 Unauthorized.

4. Kesimpulan

Berdasarkan seluruh tahapan perancangan, implementasi, pengujian, dan orkestrasi yang telah dilakukan, diperoleh beberapa kesimpulan utama:

1. **Penerapan Arsitektur Decoupled**: Pemisahan murni antara backend RESTful API (Laravel 11) dan frontend reaktif (Vue.js 3) berhasil diimplementasikan dengan kepatuhan penuh terhadap kode status HTTP semantik (201 Created, 422 Unprocessable Content, dan 401 Unauthorized) serta otentikasi token Laravel Sanctum.

2. **Jaminan Kualitas Kode Melalui Automated Testing**: Pembuatan 10 skenario *feature tests* yang lulus 100% pada eksekusi `php artisan test` menjamin seluruh logika bisnis manipulasi sumber daya terverifikasi dengan baik sebelum diintegrasikan.

3. **Kedisiplinan Alur Kerja Git dan CI/CD**: Penerapan model pencabangan `main`, `dev`, dan `feature/*` yang diproteksi oleh *Branch Protection Rules* berhasil mencegah regresi kode. Pipeline GitHub Actions secara otomatis menguji kelayakan build dan merilis citra kontainer ke *GitHub Container Registry* (GHCR) berlabel commit SHA.

4. **Efisiensi Kontainerisasi dan Portabilitas Sistem**: Penerapan *multi-stage build* sukses menghasilkan citra frontend Nginx yang sangat ringkas (≈22 MB) tanpa `node_modules`. Orkestrasi Docker Compose memungkinkan seluruh ekosistem aplikasi dijalankan secara instan dengan satu baris perintah (`docker compose up -d --build`) dan data persisten pada *named volume*.

5. **Adopsi Server Modern FrankenPHP**: Penggantian server konvensional dengan FrankenPHP (bertenaga Caddy Server) membuktikan backend Laravel dapat beroperasi dengan latensi minimal, konfigurasi terpadu, dan siap melayani kebutuhan lalu lintas produksi berkinerja tinggi.

Bibliography

[1] Bass, L., Weber, I., & Zhu, L. (2015). *DevOps: A Software Architect's Perspective*. Boston: Addison-Wesley Professional.

[2] Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures* (Doctoral dissertation). University of California, Irvine.

[3] Otwell, T., et al. (2024). *Laravel 11 Documentation: Routing, Controllers, and Sanctum Authentication*. Diakses dari `https://laravel.com/docs`.

[4] You, E., et al. (2024). *Vue.js 3 - The Progressive JavaScript Framework Documentation*. Diakses dari `https://vuejs.org/guide`.

[5] Docker Inc. (2024). *Docker Compose Specification and Best Practices for Multi-Container Applications*. Diakses dari `https://docs.docker.com/compose`.

[6] Dunglas, K., et al. (2024). *FrankenPHP: The Modern PHP App Server, Written in Go*. Diakses dari `https://frankenphp.dev/docs`.

[7] Fowler, M. (2006). *Continuous Integration*. ThoughtWorks Culture & Practices Guide.

[8] GitHub Inc. (2024). *GitHub Actions Documentation: Continuous Integration and Deployment Workflows*. Diakses dari `https://docs.github.com/en/actions`.